

Distributed Computing Principles

Distributed computing focuses on splitting large tasks into smaller subtasks that run on multiple machines simultaneously. When dealing with massive datasets, traditional single-machine clustering methods fall short. Distributed computing enables efficient processing of these datasets by splitting tasks across multiple machines. Two main advantages of splitting large tasks into smaller ones that run on independent servers or computing facilities are scalability and fault tolerance. This means that as data volumes increase, more machines can be added for processing. Also, if one server fails, others can take over to ensure continuous operation.

Importance of a Distributed System

- Scalability
 - Vertical Scaling (Scaling Up): Upgrading one machine's CPU, memory, or storage. This approach is limited by physical constraints and can become expensive quickly.
 - Horizontal Scaling (Scaling Out): Adding more machines (or nodes) to handle growing workloads. Distributed computing leverages horizontal scaling, allowing an organization to handle big datasets without relying on a single high-end server.
- Fault Tolerance and High Availability
 - Fault Tolerance: A distributed system replicates data and processes across multiple nodes, so if one node fails, others continue working.
 - High Availability: Distributing data and tasks ensures minimal downtime when a node or network link is lost.
 - Local Relevance: In Kenya, power outages and network fluctuations can disrupt standalone systems. A robust distributed setup automatically mitigates downtime.
- Resource Pooling and Load Balancing
 - Resource Aggregation: Combining the CPU, memory, and storage resources from numerous machines can be more cost-effective and powerful than a single supercomputer.
 - Load Balancing: Tasks are distributed across nodes to ensure no single machine becomes a bottleneck. This is crucial for processing massive data (for example, analyzing national-level statistics).
- Geographical Distribution
 - Distributed systems can span different counties or regions, helpful for disaster recovery and reducing latency when users are spread out.

- Kenyan Example: A bank with data centers in Nairobi and Mombasa might replicate data across both sites for local access and as a backup strategy.

Components of a Distributed System

- **Compute Nodes:** The Individual machines doing the processing or running tasks. Typically, each node has a similar hardware or software configuration, but there are cases of heterogeneous nodes with different hardware and operating systems. Nodes communicate via a network (Ethernet or fiber connections).
- **Network Infrastructure:** Connects nodes and allows them to communicate. Bandwidth (speed of data transfer) and latency (delay in communication) can significantly affect performance. In Kenya, reliable network links can be a challenge in rural regions; thus, carefully planning a cluster's network architecture is crucial).
- **Distributed Storage:** Storing data across multiple machines. Data is split into blocks or partitions and spread across multiple nodes. Replication ensures data safety and reliability. For example, HDFS (Hadoop Distributed File System).
- **Coordination and Consensus:** Mechanisms like the Zookeeper service or Raft/Paxos algorithms maintain consistent views of data and cluster state across nodes. Prevents conflicts when multiple nodes attempt to update shared resources simultaneously.

Types of Distributed Computing Architectures

1. Client-Server Model
 - Clients request services or resources; servers respond to these requests.
 - Traditional approach for web or database services.
2. Peer-to-Peer (P2P)
 - Each node has equivalent capabilities, both providing and consuming resources.
 - Popular for file-sharing networks and ad-hoc data replication.
3. Master-Slave Architecture
 - A Master node manages tasks, distributing them to Worker nodes (also called "slaves").
 - Example: Hadoop YARN, where ResourceManager acts as master and NodeManagers are workers.

Parallel vs. Distributed Computing

- **Parallel Computing** often refers to multiple threads or processes running on a single machine or tightly coupled set of machines, sharing memory.
- **Distributed Computing** involves a set of independent machines connected over a network, each with its own memory, requiring message passing to communicate.
- Big Data analytics often uses both **parallel** and **distributed** concepts. For instance, Spark clusters distribute tasks across nodes, and each node may run parallel operations on multiple CPU cores.

Practical Considerations for Distributed Systems in Kenya

1. Infrastructure Costs

- Clusters require multiple servers, storage, and reliable power and cooling.
- Economical Strategy: Start with a few nodes and scale out as needed. Public cloud solutions (AWS, Azure, GCP) may reduce capital expenses and address local power issues.

2. Network Reliability

- Some regions may experience unstable internet or power supply. Local cluster design may incorporate backup power (generators/solar panels) and robust replication strategies.

3. Local Skills and Expertise

- Building capacity: Kenyan universities are introducing specialized courses in Big Data and distributed systems, addressing the need for local expertise.
- Partnerships with local tech hubs (for example, in Nairobi's iHub ecosystem) can foster innovation and provide real-world case studies.

4. Use Cases Specific to Kenya

- Agriculture: Monitoring weather data from different counties, analyzing sensor data for irrigation.
- Finance: Processing large-scale mobile money transactions (M-Pesa) or microloan analyses.
- Healthcare: Aggregating patient data from multiple clinics to identify disease outbreaks or improve resource allocation.
- Government Services: E-citizen systems, education data management, census data analytics.

Distributed Computing in Big Data

Big Data often requires processing enormous datasets, which is where distributed computing excels. Two key frameworks are Hadoop and Apache Spark. Hadoop is an open-source framework for processing large datasets. Apache Spark is faster and more efficient than Hadoop due to in-memory computing. Ideal for real-time analytics.

1. Hadoop Ecosystem

- HDFS (Hadoop Distributed File System): Distributed storage.
- YARN (Yet Another Resource Negotiator): Cluster resource management.
- MapReduce: Batch processing engine for large-scale data tasks.

2. Apache Spark

- In-memory computation for faster data analytics.
- Rich APIs in Scala, Python, R, and Java.
- Spark SQL, Streaming, MLlib, and GraphX for diverse data processing workloads.

3. Cloud Platforms

- AWS EMR (Elastic MapReduce), Azure HDInsight, or GCP Dataproc provide managed Hadoop/Spark clusters.
- Less on-premises maintenance, potentially flexible costs (pay-as-you-go).
- More reliable infrastructure, helpful if local data centers face frequent outages.

Future of Distributed Computing

1. Serverless and Containerized Environments

- Technologies like Kubernetes and Docker are changing how distributed systems are deployed, simplifying app development and scaling.

2. Edge Computing

- Processing data near the source, beneficial for IoT or remote sensors in agriculture and smart cities.
- Reduces network overhead by performing computations on local nodes or devices.

3. AI and ML Integration

- Big Data systems provide the foundation for machine learning at scale. Tools like Spark MLlib or TensorFlow on Hadoop accelerate AI-driven solutions. Challenges in Distributed Computing

Challenges in Distributed Computing

1. Data Synchronization: Ensuring consistency across nodes.
2. Network Latency: Delays due to data transfer between nodes.
3. Fault Recovery: Quickly recovering lost data during failures.

John Swartz (2005). Big Data - Distributed Computing and Hadoop

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/PNDzhoBub00>



Tech Xplore (2022). Distributed System — Distributed Computing — Cluster Computing — Cloud Computing — Grid Computing.

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/HJiXLDIJSY>



The TechCave (2020). Distributed Systems — Distributed Computing Explained

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/ajj0EltiZm4>

